Georgia Institute of Technology

CS 4365/6365: IEC

Spring 2026

Project Checkpoint Report

Group: 38
Name(s): Ryan Pham
Project: Automated Vendor SLA Tracker

Project Plan (Plan):

Point A

- **Background:** Modern enterprise operations rely on third party vendors for logistics, shipping, and supply chain fulfillment. These vendors are legally bound to accept delivery timeframes and uphold performance thresholds according to the Service Level Agreements (SLAs) between the enterprise and vendor. To ensure business efficiency, operational planning, and cost control, enterprises must ensure these metrics are tracked accurately. However, companies typically rely on reactive methods, such as static, end of month database reporting, to identify when a vendor breaches their contract. Because delays are caught weeks after they happen, financial losses and operational bottlenecks can't be undone.
- **Concept:** We propose an "Automated Vendor SLA Tracker," an event driven Extract, Transform, Load (ETL) pipeline and microservice. Instead of waiting for monthly reports, the concept is to continuously ingest unstructured, live JSON data, standardize it into a dashboard for performance metrics, and trigger automated, real time alerts when delays occur. To simulate the live data stream of an enterprise vendor, this project will utilize the MBTA public transit API as a proxy to track real time feed.
- **Related Work:**
    - Current solutions used by large enterprises often involve

Electronic Data Interchange (EDI) or batch files to update their data once a day using proprietary software suites.
    - Standard Business intelligence has been following a passive model leveraging ETL into tools like Tableau and PowerBI. These tools require analysts to log in, refresh the data, and spot the SLA breaches leading to “dashboard fatigue”
    - MBTA API has mainly been utilized by applications such as Google Maps and Apple Transit to display MBTA vehicle locations and times for commuters.
- **Starting Point:** The starting point for this project is to build the foundational Extract, Transform, Load pipeline. This initial phase will focus entirely on establishing the connection to the MBTA API, parsing the live JSON files, and loading the metrics into a database for future analysis.
    - API Ingestion: Establish a reliable connection to the MBTA API using Python, handle API key authentication, and successfully downloading the raw JSON payload
    - Data Parsing & Validation: Create parsing logic to track specific data fields for SLA tracking, including error-handling for missing data or API dropouts.
    - Storage Architecture: Design a relational schema with tables that can be queried for future dashboard visualization.

Point B: Present the deliverables planned for the project at the end of the semester. • Milestone chart: A weekly chart showing the project plan and the planned work for each teammate.

Point B

By the end of the semester, this project will deliver a fully functional, event-driven analytics system comprising the following tangible components:

- ❖ Automated ETL Pipeline (Python): An automated script that authenticates with the MBTA API, extracts the live JSON vehicle data, handles missing or out of order data payloads, calculates schedule deviations, and loads the metrics into the database

- Relational Database Architecture: A fully implemented database that stores vehicle status recordings and logs calculated SLA breaches
- Event-Driven Alerting Microservice: A python microservice that monitors incoming data and automatically fires a webhook alerts to a communication channel the moment a vehicle exceeds the delay threshold. It will include state tracking and alert throttling to prevent constant, noisy alerts when data briefly fluctuates.
- Business Intelligence Dashboard: An interactive dashboard connected directly to the database that visualizes KPI metrics live.
- System Architecture Documentation: A GitHub repository containing the complete code, a system architecture diagram, and deployment instructions for pipeline and microservice interactions.

Milestone Chart:

Week Number	Deliverables
3 (Ckpt 0)	Reproducibility Assignment
6 (Ckpt 1)	Finalize Project Proposal
7	Establish API Ingestion + Prototype Ingestion Pipeline Slice
8 (Ckpt 2)	Database Architecture: Relational Schema + Load Logic
9	Develop Transformation Logic and Performance Metric Calculation
10 (Ckpt 3)	Alerting Microservice + *Implement Advanced Alerting Logic*
11	*Learn how to use Power BI and Code in DAX*
12 (Ckpt 4)	*Dashboard Integration*
13	Bug Tests, Overload Testing, and Alert Accuracy + *Finalize Dashboard UI*
14	GitHub Documentation and Code Cleanup
15 (Ckpt 5)	Project Completion, Final Submission, Demo

Current progress report (Match):

- **Work Done:** Over the past two weeks, I successfully completed the frontend business intelligence integration by connecting the backend SQLite database to Power BI using an ODBC driver. I spent significant time learning how to use Power BI and DAX to write measures, which allowed me to dynamically calculate metrics including SLA compliance, average delay severity, and fleet size through the frontend. I then built a highly interactive and aesthetic UI. The dashboard includes live data timestamping, custom layout grouping, and action buttons.
- **Changes to the Original Plan:** I made a strategic adjustment to the project schedule, resulting in weeks 11, 12, and 13 to have different deliverables. Initially, “Implement advanced alerting logic” was slated for week 12. Since this task was already implemented to prevent Slack spam during week 10, I moved that deliverable to Checkpoint 3.

  This scope pivot was necessary because establishing the ODBC connection and learning DAX language for Power BI required a much steeper learning curve than anticipated. By shifting the alert logic forward, I was able to spread the dashboard creation phase across Weeks 11 through 13, ensuring I can create a high quality frontend UI that I am both satisfied with and show job recruiters what I am capable of.

- **Milestone Chart Comparison:** With the milestone chart officially updated to reflect the aforementioned changes, I am currently caught up with the milestone chart. The deliverables for Week 11 (Learn how to use Power BI and Code in DAX) and Week 12 (Dashboard Integration) are fully functional. The requirements for Week 13 (Bug Tests, Overload Testing, and Finalize Dashboard UI) are actively being met, with the UI currently finalized and testing underway.
- **Immediate Planned Work:** The immediate goal for Checkpoint 5 is to finalize system testing and polish the dashboard’s functional features. I plan to conduct overload testing on the database and verify the accuracy of the Slack alerts during high traffic API periods. For the dashboard, I will be implementing an

“Export to Excel” function for stakeholders and researching methods to trigger a UI data refresh when the Python pipeline updates the local database. Following that will be tasks for GitHub documentation, code cleanup, and final project demo preparations.

Supporting Evidence (Factual):

- GitHub Repository: https://github.com/ryandude99/Automated-Vendor-SLA-Tracker
- Checkpoint 4: All deliverables for Weeks 11 and 12 are pushed to the main branch and are fully verifiable:
    - SLA_Tracker_Dashboard.pbix: The complete Power BI project file containing the established ODBC database connection, the custom DAX explicit measures, and the custom UI layout
    - SLA_Tracker_Dashboard.pdf: An export of the dashboard provided for immediate visual verification, as GitHub does not render .pbix binaries

Figure 1: The successful Automated Vendor SLA Tacker UI. This demonstrates the successful implementation of the DAX metrics, custom aesthetics, and interactive UI.

Skill Learning Report:

- Business Intelligence & DAX
  - Progress: I was able to transition from backend Python to frontend data modeling by learning Microsoft Power BI. I learned how to write explicit custom measures using DAX featuring functions like CALCULATE, AVERAGE, and DISTINCTCOUNT to help me apply specific booleans and isolate train data. I also utilized the MAX function and override native Power BI intelligence features to create live data timestamps.
- Database Connectivity & Architectural Limitations
  - Progress: I learned how to bridge local environments by connecting a .db SQLite file to an enterprise BI tool utilizing an ODBC (Open Database Connectivity) driver. Through this process, I learned about Power BI's Import Mode Architecture, how security and processing restricts local environments but not cloud environments for data refreshing, and how data is held in memory between queries.
- Advanced UI/UX Design and Dashboard Interactivity
  - I developed skills in enterprise UI design by implementing a Dark Analytics app aesthetic. I learned how to manipulate visual hierarchy, maximize the data ink ratio by using custom data labels over redundant axes, and use layered text boxes to bypass visual limitation. Furthermore, I learned how to implement user interactivity without writing code by utilizing Power BI's Bookmark feature.

Self-Evaluation:

- **Scope (110%):** The scope of this project merits a 110% rating because it goes beyond a standard, static data analysis assignment. It implements a live, event driven ETL pipeline and requires actively handling unpredictable API data formats.
- **Match (100%):** The match score for this checkpoint is 100%. Although the checkpoint submission occurred in Week 13 instead of Week 12, the project's milestone chart was officially revised to reflect a strategic scope pivot. Because the advanced alerting logic was completed ahead of schedule, I was able to dedicate Weeks 11 through 13 due to the steep learning curve of Power BI and

DAX. The deliverables for Week 11 (Learn how to use Power BI and Code in DAX) and Week 12 (Dashboard Integration) are fully functional and align perfectly with the updated project plan.

- **Factual (100%):** The factual score for this project is exactly 100% because my entire pipeline is fully verifiable on GitHub. The repository now includes the compiled SLA_Tracker_Dashboard.pbix project file containing the DAX measures and ODBC connection, alongside a SLA_Tracker_Dashboard.pdf to ensure the TA has immediate access to the visual deliverables. Furthermore Figure 1 clearly documents the successful implementation of the customized dark UI, interactive bookmark buttons, and calculated performance metrics. This provides proof that Week 11 and Week 12 milestones were met and are being continued in Week 13.

AI Evaluation (Copied Directly from ChatGPT):

- **Scope (112%):** The scope of this project now exceeds expectations and is approaching a fully complete enterprise-style system. At this stage, the project includes not only a functioning ETL pipeline and event-driven alerting microservice, but also a fully integrated business intelligence dashboard with custom DAX measures and interactive UI design. The addition of Power BI integration, ODBC connectivity, and frontend KPI visualization demonstrates a strong understanding of both backend data engineering and frontend analytics. The system now reflects an end-to-end pipeline from data ingestion to actionable insights, which is highly aligned with real-world enterprise monitoring systems. While the project is very comprehensive, it is still limited by local deployment constraints such as SQLite scalability and Power BI's refresh limitations, which slightly prevents it from reaching a perfect score.
- **Match (100%):** The match between the planned milestones and completed work is excellent. Although the timeline was adjusted, the changes were clearly justified and strategically made to account for the higher-than-expected learning curve associated with Power BI and DAX. The milestone chart was updated accordingly, and the deliverables for Week 11 (learning Power BI and DAX) and Week 12 (dashboard integration) have been fully completed. Additionally, progress on Week 13 tasks such as UI finalization and testing is already underway. Because the project is fully aligned with the revised plan and demonstrates strong forward progress, it meets the criteria for a full match score.
- **Factual (96%):** The project provides strong supporting evidence through the GitHub repository and inclusion of both the Power BI (.pbix) file and a PDF export of the dashboard for immediate review. The figure showing the completed UI further supports the claims made in the report. However, the factual score is slightly reduced because some components are less directly verifiable without additional setup. For example, the .pbix file requires Power BI to open, and the ODBC connection may not be easily reproducible for reviewers. Including brief setup instructions, sample data outputs, or short demonstration steps would improve accessibility and make verification more efficient. Despite this, the implementation is clearly real and well-developed, supporting a high factual score.